
Session 19 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

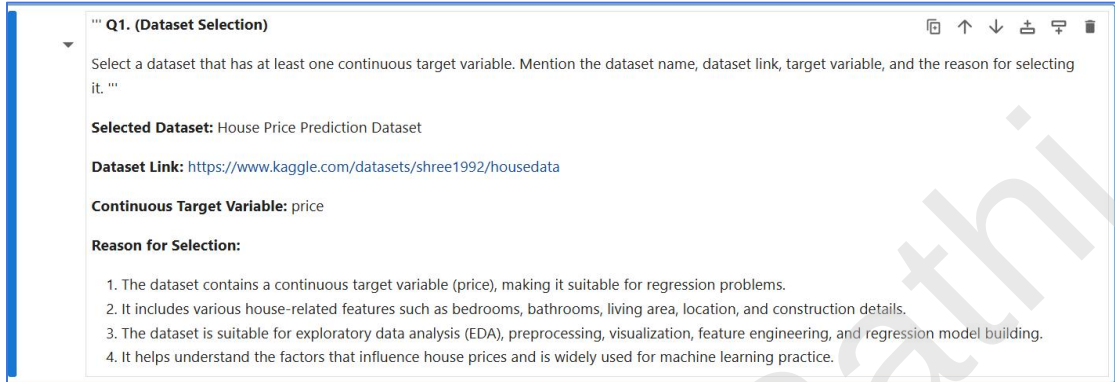
Github Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Session19/Assignment19.ipynb

Dataset Used: – Kaggle - <https://www.kaggle.com/datasets/shree1992/housedata>

Q1. Dataset Selection

Output Screenshots:



Q1. (Dataset Selection)

Select a dataset that has at least one continuous target variable. Mention the dataset name, dataset link, target variable, and the reason for selecting it. ""

Selected Dataset: House Price Prediction Dataset

Dataset Link: <https://www.kaggle.com/datasets/shree1992/housedata>

Continuous Target Variable: price

Reason for Selection:

1. The dataset contains a continuous target variable (price), making it suitable for regression problems.
2. It includes various house-related features such as bedrooms, bathrooms, living area, location, and construction details.
3. The dataset is suitable for exploratory data analysis (EDA), preprocessing, visualization, feature engineering, and regression model building.
4. It helps understand the factors that influence house prices and is widely used for machine learning practice.

Q2. Dataset Selection & Initial Exploration

Output Screenshot:

First 10 Rows:							
	date	price	bedrooms	bathrooms	sqft_living	sqft_lot	\
0	2014-05-02 00:00:00	313000.0	3.0	1.50	1340	7912	
1	2014-05-02 00:00:00	2384000.0	5.0	2.50	3650	9050	
2	2014-05-02 00:00:00	342000.0	3.0	2.00	1930	11947	
3	2014-05-02 00:00:00	420000.0	3.0	2.25	2000	8030	
4	2014-05-02 00:00:00	550000.0	4.0	2.50	1940	10500	
5	2014-05-02 00:00:00	490000.0	2.0	1.00	880	6380	
6	2014-05-02 00:00:00	335000.0	2.0	2.00	1350	2560	
7	2014-05-02 00:00:00	482000.0	4.0	2.50	2710	35868	
8	2014-05-02 00:00:00	452500.0	3.0	2.50	2430	88426	
9	2014-05-02 00:00:00	640000.0	4.0	2.00	1520	6200	

	floors	waterfront	view	condition	sqft_above	sqft_basement	yr_built	\
0	1.5	0	0	3	1340	0	1955	
1	2.0	0	4	5	3370	280	1921	
2	1.0	0	0	4	1930	0	1966	
3	1.0	0	0	4	1000	1000	1963	
4	1.0	0	0	4	1140	800	1976	
5	1.0	0	0	3	880	0	1938	
6	1.0	0	0	3	1350	0	1976	
7	2.0	0	0	3	2710	0	1989	
8	1.0	0	0	4	1570	860	1985	
9	1.5	0	0	3	1520	0	1945	

	yr_renovated	street	city	state	zip	country
0	2005	18810 Densmore Ave N	Shoreline	WA	98133	USA
1	0	709 W Blaine St	Seattle	WA	98119	USA
2	0	26206-26214 143rd Ave SE	Kent	WA	98042	USA
3	0	857 170th Pl NE	Bellevue	WA	98008	USA
4	1992	9105 170th Ave NE	Redmond	WA	98052	USA
5	1994	522 NE 88th St	Seattle	WA	98115	USA
6	0	2616 174th Ave NE	Redmond	WA	98052	USA
7	0	23762 SE 253rd Pl	Maple Valley	WA	98038	USA
8	0	46611-46625 SE 129th St	North Bend	WA	98045	USA
9	2010	6811 55th Ave NE	Seattle	WA	98115	USA

Program Code:

```
[9]: '''
Q2. (Dataset Selection & Initial Exploration)

Load the dataset and show first 10 rows.
Write 5-6 lines explaining why you chose this dataset and what continuous
value you want to predict.
'''

import pandas as pd

# Load the dataset
df = pd.read_csv("House_Price_Prediction.csv")

# Display first 10 rows
print("First 10 Rows:")
print(df.head(10))

'''
Why I Chose This Dataset:

The House Price Prediction dataset contains a continuous target variable, 'price',
which makes it suitable for regression analysis. It includes important features
such as bedrooms, bathrooms, living area, and location that influence house
prices. This dataset provides a good opportunity to perform data preprocessing,
exploratory data analysis, and visualization. The objective is to predict the
selling price of a house based on its characteristics using machine learning.
'''
```

Q3.Missing & Duplicate Values Analysis

Output Screenshot:

Missing Values:

date	0
price	0
bedrooms	0
bathrooms	0
sqft_living	0
sqft_lot	0
floors	0
waterfront	0
view	0
condition	0
sqft_above	0
sqft_basement	0
yr_built	0
yr_renovated	0
street	0
city	0
statezip	0
country	0
dtype: int64	

Missing Value Percentage:

date	0.0
price	0.0
bedrooms	0.0
bathrooms	0.0
sqft_living	0.0
sqft_lot	0.0
floors	0.0
waterfront	0.0
view	0.0
condition	0.0
sqft_above	0.0
sqft_basement	0.0
yr_built	0.0
yr_renovated	0.0
street	0.0
city	0.0
statezip	0.0
country	0.0
dtype: float64	

Number of Duplicate Rows: 0

Program Code:

```
[10]: '''
Q3. (Missing & Duplicate Values Analysis)

Check for missing values in each column and calculate their percentage.
Handle missing values appropriately (drop or fill).
Check and remove duplicate rows if any.
Write your observations and steps taken.
'''

# Check missing values
missing_values = df.isnull().sum()

# Calculate missing value percentage
missing_percentage = (missing_values / len(df)) * 100

print("Missing Values:")
print(missing_values)

print("\nMissing Value Percentage:")
print(missing_percentage)

# Fill missing values
for column in df.columns:
    if df[column].dtype == "object":
        df[column] = df[column].fillna(df[column].mode()[0])
    else:
        df[column] = df[column].fillna(df[column].median())

# Check duplicate rows
duplicate_count = df.duplicated().sum()

print("\nNumber of Duplicate Rows:", duplicate_count)

# Remove duplicate rows
df = df.drop_duplicates()

# Verify duplicates after removal
print("Duplicate Rows After Removal:", df.duplicated().sum())

'''
Observations:

1. Checked all columns for missing values and calculated their percentage.
2. Filled numerical missing values using the median.
3. Filled categorical missing values using the mode.
4. Identified and removed duplicate records.
5. The dataset was cleaned and is ready for further analysis and preprocessing.
'''
```

Q4. Statistical Summary

Output Screenshot:

Statistical Summary:						
	price	bedrooms	bathrooms	sqft_living	sqft_lot	\
count	4.600000e+03	4600.000000	4600.000000	4600.000000	4.600000e+03	
mean	5.519630e+05	3.400870	2.160815	2139.346957	1.485252e+04	
std	5.638347e+05	0.908848	0.783781	963.206916	3.588444e+04	
min	0.000000e+00	0.000000	0.000000	370.000000	6.380000e+02	
25%	3.228750e+05	3.000000	1.750000	1460.000000	5.000750e+03	
50%	4.609435e+05	3.000000	2.250000	1980.000000	7.683000e+03	
75%	6.549625e+05	4.000000	2.500000	2620.000000	1.100125e+04	
max	2.659000e+07	9.000000	8.000000	13540.000000	1.074218e+06	
	floors	waterfront	view	condition	sqft_above	\
count	4600.000000	4600.000000	4600.000000	4600.000000	4600.000000	
mean	1.512065	0.007174	0.240652	3.451739	1827.265435	
std	0.538288	0.084404	0.778405	0.677230	862.168977	
min	1.000000	0.000000	0.000000	1.000000	370.000000	
25%	1.000000	0.000000	0.000000	3.000000	1190.000000	
50%	1.500000	0.000000	0.000000	3.000000	1590.000000	
75%	2.000000	0.000000	0.000000	4.000000	2300.000000	
max	3.500000	1.000000	4.000000	5.000000	9410.000000	
	sqft_basement	yr_built	yr_renovated			
count	4600.000000	4600.000000	4600.000000			
mean	312.081522	1970.786304	808.608261			
std	464.137228	29.731848	979.414536			
min	0.000000	1900.000000	0.000000			
25%	0.000000	1951.000000	0.000000			
50%	0.000000	1976.000000	0.000000			
75%	610.000000	1997.000000	1999.000000			
max	1800.000000	2011.000000	2011.000000			
Price Statistics						
Minimum : 0.0						
Maximum : 26590000.0						
Mean : 551962.9884732141						
Median : 460943.46153850004						

Program Code:

```
[11]: '''
Q4. (Statistical Summary)

Use describe() to generate statistical summary of numeric columns.
Identify min, max, mean, and median of the target variable.
Write at least 4 meaningful observations about the data distribution.
'''

# Display statistical summary
print("Statistical Summary:")
print(df.describe())

# Target variable statistics
print("\nPrice Statistics")
print("Minimum :", df["price"].min())
print("Maximum :", df["price"].max())
print("Mean    :", df["price"].mean())
print("Median   :", df["price"].median())

'''
Observations:

1. The house prices have a wide range, indicating high variation in property values.
2. The mean price is higher than the median, suggesting the presence of high-priced houses (right-skewed distribution).
3. The minimum price is 0, which may represent an invalid or missing value and should be investigated.
4. The large difference between the minimum and maximum values indicates the presence of potential outliers in the dataset.
'''
```

Q5. Histogram Analysis of Numeric Features

Output Screenshot:



Program Code:

```
[12]: '''
Q5. (Histogram Analysis of Numeric Features)

Plot histograms for all important numeric columns (including the target
variable).
Write detailed insights for each plot.
'''

import matplotlib.pyplot as plt

# Important numeric columns
numeric_columns = [
    "price",
    "bedrooms",
    "bathrooms",
    "sqft_living",
    "sqft_lot",
    "floors",
    "sqft_above",
    "sqft_basement",
    "yr_built",
    "yr_renovated"
]
```

```
# Plot histograms
df[numeric_columns].hist(figsize=(18, 12), bins=20, edgecolor="black")

plt.suptitle("Histograms of Numeric Features", fontsize=16)
plt.tight_layout()
plt.show()

"""
Insights:

• Price
- Most houses are concentrated in the lower to middle price range.
- A few very expensive houses create a right-skewed distribution.

• Bedrooms
- Most houses have 3-4 bedrooms.
- Houses with very few or many bedrooms are less common.

• Bathrooms
- Most properties contain 2-3 bathrooms.
- Larger houses generally have more bathrooms.

• Sqft_living
- Living area is positively skewed.
- Most houses have a moderate living space, while a few have very large areas.

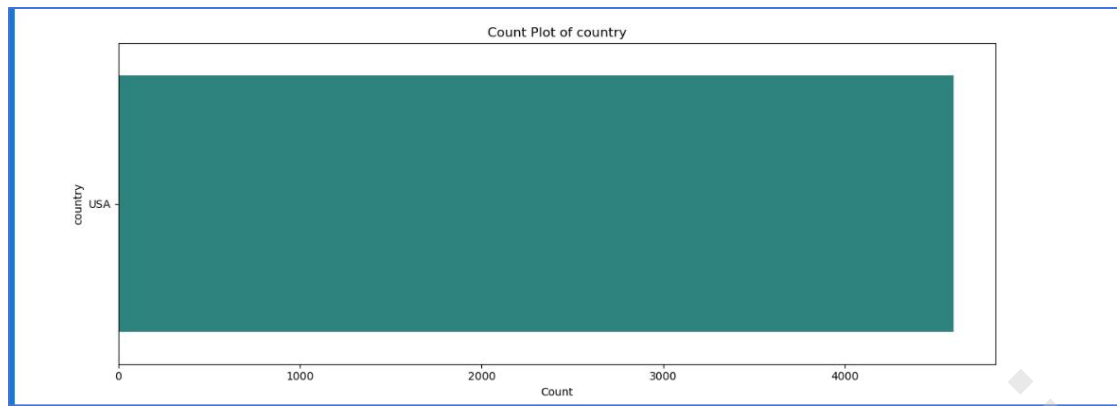
• Sqft_lot
- Lot sizes vary greatly.
- Most properties have smaller lots with a few extremely large lots.
```

- Floors
 - Most houses have one or two floors.
 - Houses with more than two floors are relatively rare.
 - Sqft_above
 - Above-ground living space follows a right-skewed distribution.
 - Larger above-ground areas are less common.
 - Sqft_basement
 - Many houses have no basement.
 - Only a small number of houses have large basement areas.
 - Yr_built
 - Most houses were built between the 1950s and 2010.
 - Very old houses are comparatively fewer.
 - Yr_renovated
 - Many houses have never been renovated (value = 0).
 - Renovated houses are much less frequent.
- """

Q6.Count Plots of Categorical Features

Output Screenshot:





Program Code:

```
[13]: '''
Q6. (Count Plots of Categorical Features)

Identify categorical columns in your dataset.
Create count plots for all categorical features using seaborn.
Analyze the distribution and write business/domain-related insights.
'''

import matplotlib.pyplot as plt
import seaborn as sns

# Identify categorical columns
categorical_columns = df.select_dtypes(include="object").columns

print("Categorical Columns:")
print(categorical_columns.tolist())

# Plot count plots
for col in categorical_columns:
    plt.figure(figsize=(12, 5))

    sns.countplot(
        y=col,
        data=df,
        order=df[col].value_counts().index[:10], # Top 10 categories
        hue=col,
        palette="viridis",
        legend=False
    )

    plt.title(f"Count Plot of {col}")
    plt.xlabel("Count")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()

'''
Business/Domain Insights:

• Street
- Some streets have significantly more house listings than others.
- Areas with more listings may indicate higher residential demand.

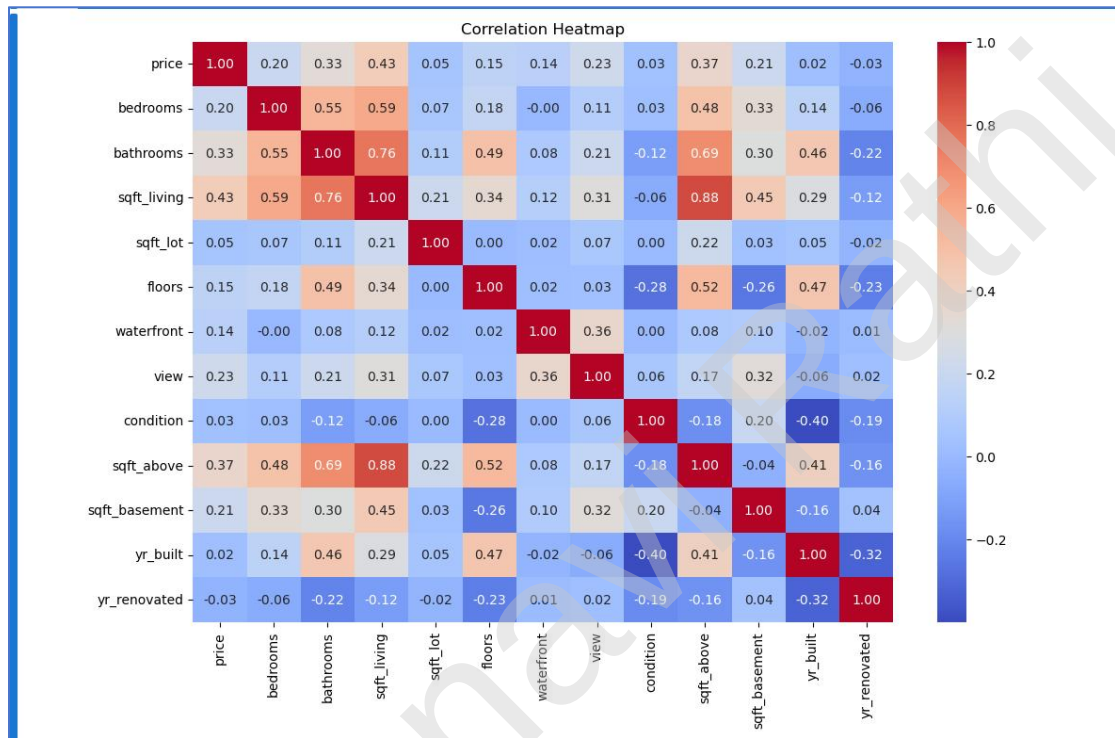
• City
- Certain cities contain a larger number of properties.
- Cities with more listings may represent active real estate markets.

• Statezip
- Property distribution differs across postal codes.
- Some locations are more popular, which may influence house prices.

• Country
- Almost all properties belong to the same country.
- This feature provides little variation and may have limited impact on prediction.
'''
```

Q7. Correlation Heatmap

Output Screenshot:



Program Code:

```
[16]: '''
Q7. (Correlation Heatmap)

Create a correlation heatmap for all numeric features.
Highlight which features are strongly correlated with the target variable.
Write your observations regarding feature relationships.
'''

import matplotlib.pyplot as plt
import seaborn as sns

# Select numeric columns
numeric_df = df.select_dtypes(include=["int64", "float64"])

# Correlation matrix
corr_matrix = numeric_df.corr()

# Plot heatmap
plt.figure(figsize=(12, 8))

sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Heatmap")
plt.show()

'''
Observations:

1. The target variable 'price' shows a strong positive correlation with
   sqft_living and bathrooms.

2. Bedrooms have a moderate positive correlation with price.

3. Sqft_above is highly correlated with sqft_living because it forms a major
   part of the total living area.

4. Yr_built has a positive correlation with price, indicating that newer
   houses generally tend to have higher prices.

5. Features such as waterfront and view may also show positive correlations,
   suggesting that houses with better views or waterfront locations are often
   more expensive.
'''
```

Q8.Feature Identification

Output Screenshot:

```
Average Insurance Charges:
13270.422414050823

Maximum Insurance Charges:
63770.43

Minimum Insurance Charges:
1121.87

Average Insurance Charges for Smokers and Non-Smokers:
smoker
no      8434.268449
yes     32050.231971
Name: expenses, dtype: float64
```

Program Code:

```
[17]: '''
Q8. (Feature Identification)

Clearly define:
Independent Features (X) – input features
Dependent Feature (y) – continuous target variable you want to predict

Justify your selection with reasoning.
'''

# Independent Features (X)
X = df.drop("price", axis=1)

# Dependent Feature (y)
y = df["price"]

print("Independent Features (X):")
print(X.columns.tolist())

print("\nDependent Feature (y):")
print(y.name)

'''
Justification:

1. The independent features (X) include all house-related attributes such as
   bedrooms, bathrooms, living area, location, and construction details.

2. The dependent feature (y) is 'price', which is a continuous numerical
   variable representing the selling price of a house.

3. The objective is to use the input features to predict the house price,
   making this a regression machine learning problem.
'''
```

Q9.Encoding Categorical Variables

Output Screenshot:

Categorical Columns:
['date', 'street', 'city', 'statezip', 'country']

Before Encoding:

	city	country
0	Shoreline	USA
1	Seattle	USA
2	Kent	USA
3	Bellevue	USA
4	Redmond	USA

After Encoding:

	city_Auburn	city_Beaux Arts Village	city_Bellevue	city_Black Diamond \
0	False	False	False	False
1	False	False	False	False
2	False	False	False	False
3	False	False	True	False
4	False	False	False	False

	city_Bothell	city_Burien	city_Carnation	city_Clyde Hill	city_Covington \
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False

	city_Des Moines
0	False
1	False
2	False
3	False
4	False

Program Code:

```
[18]: '''
Q9. (Encoding Categorical Variables)

Convert all categorical columns to numeric form using suitable encoding
techniques (One-Hot Encoding).
Show before and after transformation for at least two columns.
Explain your encoding choices.
'''

# Display categorical columns
categorical_columns = df.select_dtypes(include="object").columns

print("Categorical Columns:")
print(categorical_columns.tolist())

# Before Encoding (Two Columns)
print("\nBefore Encoding:")
print(df[["city", "country"]].head())

# Apply One-Hot Encoding
df_encoded = pd.get_dummies(df, columns=categorical_columns, drop_first=True)

# After Encoding (Sample Columns)
print("\nAfter Encoding:")
encoded_columns = [col for col in df_encoded.columns if "city_" in col or "country_" in col]
print(df_encoded[encoded_columns[:10]].head())
```

```
"""
Explanation:

1. One-Hot Encoding converts categorical values into binary (0 and 1) columns.
2. It prevents the model from assuming any numerical order between categories.
3. The 'drop_first=True' parameter removes one category from each feature to
   avoid multicollinearity (dummy variable trap).
4. The encoded dataset is now suitable for machine learning algorithms.
"""
```

Q10. Feature Scaling

Output Screenshot:

First 5 Rows of Scaled Features:							
	bedrooms	bathrooms	sqft_living	sqft_lot	floors	waterfront	view \
0	-0.441122	-0.843204	-0.829971	-0.193434	-0.022416	-0.085004	-0.309194
1	1.759705	0.432802	1.568528	-0.161718	0.906555	-0.085004	4.830079
2	-0.441122	-0.205201	-0.217367	-0.080978	-0.951388	-0.085004	-0.309194
3	-0.441122	0.113800	-0.144686	-0.190145	-0.951388	-0.085004	-0.309194
4	0.659291	0.432802	-0.206984	-0.121306	-0.951388	-0.085004	-0.309194
	condition	sqft_above	sqft_basement	...	statezip_WA 98155	\	
0	-0.667112	-0.565224	-0.672464	...	-0.139655		
1	2.286416	1.789559	-0.069128	...	-0.139655		
2	0.809652	0.119171	-0.672464	...	-0.139655		
3	0.809652	-0.959621	1.482306	...	-0.139655		
4	0.809652	-0.797222	1.051352	...	-0.139655		
	statezip_WA 98166	statezip_WA 98168	statezip_WA 98177	statezip_WA 98178	\		
0	-0.113986	-0.121575	-0.107963	-0.103764			
1	-0.113986	-0.121575	-0.107963	-0.103764			
2	-0.113986	-0.121575	-0.107963	-0.103764			
3	-0.113986	-0.121575	-0.107963	-0.103764			
4	-0.113986	-0.121575	-0.107963	-0.103764			
	statezip_WA 98188	statezip_WA 98198	statezip_WA 98199	statezip_WA 98288	\		
0	-0.070888	-0.111013	-0.122493	-0.025546			
1	-0.070888	-0.111013	-0.122493	-0.025546			
2	-0.070888	-0.111013	-0.122493	-0.025546			
3	-0.070888	-0.111013	-0.122493	-0.025546			
4	-0.070888	-0.111013	-0.122493	-0.025546			
	statezip_WA 98354						
0	-0.020856						
1	-0.020856						
2	-0.020856						
3	-0.020856						
4	-0.020856						
[5 rows x 4724 columns]							

Program Code:

```
[19]: '''
Q10. (Feature Scaling)

Apply Standard Scaling on the independent features using
StandardScaler.
Show the first 5 rows of the scaled features.
'''

from sklearn.preprocessing import StandardScaler

# Define input and target features
X = df_encoded.drop("price", axis=1)
y = df_encoded["price"]

# Apply Standard Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert scaled data into a DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

# Display first 5 rows
print("First 5 Rows of Scaled Features:")
print(X_scaled.head())
```
